

Lý thuyết: RFM Analysis - Phương pháp Phân nhóm Khách hàng

Mục lục

1. Tổng quan về RFM Analysis

- 1.1. RFM là gì?
- 1.2. Tại sao cần RFM?
- 1.3. Ứng dụng thực tế

2. Cơ sở dữ liệu và Tính toán RFM

- 2.1. Dữ liệu cần thiết
- 2.2. Tính Recency (R)
- 2.3. Tính Frequency (F)
- 2.4. Tính Monetary (M)

3. Phương pháp 1: RFM Score (Quintile)

- 3.1. Cách tính RFM Score
- 3.2. Phân nhóm khách hàng
- 3.3. Ưu điểm
- 3.4. Nhược điểm
- 3.5. Case ứng dụng

4. Phương pháp 2: K-means Clustering

- 4.1. Tổng quan về K-means
- 4.2. Chuẩn hóa dữ liệu RFM
- 4.3. Xác định số cluster (K)
- 4.4. Phân nhóm bằng K-means
- 4.5. Ưu điểm
- 4.6. Nhược điểm
- 4.7. Case ứng dụng

5. So sánh 2 Phương pháp

- 5.1. Bảng so sánh chi tiết
- 5.2. Khi nào dùng phương pháp nào?

6. Chuẩn hóa Feature cho RFM

- 6.1. Tại sao cần chuẩn hóa?
- 6.2. Min-Max Scaling
- 6.3. Z-score Normalization (Standardization)
- 6.4. Robust Scaling

7. Visualization

- 7.1. RFM Heatmap
- 7.2. Segment Profile
- 7.3. Cluster Visualization

1. Tổng quan về RFM Analysis

1.1. RFM là gì?

RFM là phương pháp phân tích khách hàng dựa trên 3 chỉ số hành vi:

- **R (Recency):** Khoảng thời gian từ lần giao dịch cuối cùng đến hiện tại
 - Đo lường: Số ngày/tuần/tháng
 - Ý nghĩa: Khách hàng giao dịch càng gần đây → Càng tích cực
- **F (Frequency):** Số lần giao dịch trong một khoảng thời gian quan sát
 - Đo lường: Số lần (count)
 - Ý nghĩa: Khách hàng giao dịch càng nhiều → Càng trung thành
- **M (Monetary):** Tổng giá trị giao dịch trong khoảng thời gian đó
 - Đo lường: Tổng số tiền (VND, USD, ...)
 - Ý nghĩa: Khách hàng chi tiêu càng nhiều → Càng có giá trị

1.2. Tại sao cần RFM?

1. **Phân nhóm khách hàng tự động:** Không cần kiến thức chuyên sâu về business
2. **Dựa trên hành vi thực tế:** Phản ánh đúng hành vi giao dịch của khách hàng
3. **Dễ hiểu và triển khai:** Logic đơn giản, kết quả dễ giải thích
4. **Ứng dụng marketing hiệu quả:** Mỗi nhóm có chiến lược riêng
5. **Đo lường được:** Có thể đánh giá hiệu quả chiến dịch

1.3. Ứng dụng thực tế

- **E-commerce/Retail:** Phân nhóm khách hàng mua sắm
- **Banking/Fintech:** Đánh giá giá trị khách hàng, xác định hạn mức
- **Telecom:** Phân nhóm thuê bao, chiến lược retention
- **SaaS:** Phân nhóm user theo mức độ sử dụng
- **Hospitality:** Phân nhóm khách hàng thường xuyên

2. Cơ sở dữ liệu và Tính toán RFM

2.1. Dữ liệu cần thiết

Để tính RFM, cần có bảng giao dịch với các trường:

Trường	Mô tả	Ví dụ
customer_id	Mã định danh khách hàng	"C001", "CUST_1234"

Trường	Mô tả	Ví dụ
transaction_date	Ngày giao dịch	"2024-01-15", "2024-12-01"
amount	Giá trị giao dịch	500000, 1000000

Ví dụ dữ liệu:

```
# Dữ liệu giao dịch mẫu
import pandas as pd

transactions = pd.DataFrame({
    'customer_id': ['C001', 'C001', 'C002', 'C003', 'C001'],
    'transaction_date': ['2024-01-15', '2024-02-20', '2024-03-10', '2024-11-25',
                        '2024-10-05'],
    'amount': [500000, 1000000, 200000, 5000000, 300000]
})

transactions['transaction_date'] =
pd.to_datetime(transactions['transaction_date'])
```

2.2. Tính Recency (R)

Recency = Số ngày từ lần giao dịch cuối cùng đến ngày phân tích

```
# Ngày phân tích (thường là ngày hiện tại hoặc ngày cuối cùng có dữ liệu)
analysis_date = pd.Timestamp('2024-12-01')

# Tính Recency cho từng khách hàng
recency = transactions.groupby('customer_id')['transaction_date'].max()
recency = (analysis_date - recency).dt.days

print(recency)
# Output:
# customer_id
# C001      62   (giao dịch cuối: 2024-10-05)
# C002     266   (giao dịch cuối: 2024-03-10)
# C003       6   (giao dịch cuối: 2024-11-25)
```

Lưu ý:

- R **càng nhỏ** = Giao dịch càng gần đây = **Tốt hơn**
- R **càng lớn** = Lâu không giao dịch = **Xấu hơn**

2.3. Tính Frequency (F)

Frequency = Số lần giao dịch của khách hàng trong khoảng thời gian quan sát

```
# Tính Frequency
frequency = transactions.groupby('customer_id')['transaction_date'].count()

print(frequency)
# Output:
# customer_id
# C001      3
# C002      1
# C003      1
```

Lưu ý:

- **F càng lớn** = Giao dịch càng nhiều = **Tốt hơn**

2.4. Tính Monetary (M)

Monetary = Tổng giá trị giao dịch của khách hàng trong khoảng thời gian quan sát

```
# Tính Monetary
monetary = transactions.groupby('customer_id')['amount'].sum()

print(monetary)
# Output:
# customer_id
# C001      1800000    (500K + 1M + 300K)
# C002       200000
# C003      5000000
```

Lưu ý:

- **M càng lớn** = Chi tiêu càng nhiều = **Tốt hơn**

2.5. Tổng hợp bảng RFM

```
# Tạo bảng RFM
rfm_df = pd.DataFrame({
    'recency': recency,
    'frequency': frequency,
    'monetary': monetary
}).reset_index()

print(rfm_df)
# Output:
#   customer_id  recency  frequency  monetary
# 0          C001       62          3  1800000
# 1          C002      266          1   200000
# 2          C003        6          1  5000000
```

3. Phương pháp 1: RFM Score (Quintile)

3.1. Cách tính RFM Score

Phương pháp **RFM Score** chia khách hàng thành **5 nhóm (quintile)** cho mỗi chỉ số, sau đó kết hợp thành điểm số.

Bước 1: Tính điểm cho từng chỉ số (1-5)

```
# Tính R_score (1-5)
# Lưu ý: Recency càng nhỏ càng tốt, nên đảo ngược
rfm_df['R_score'] = pd.qcut(rfm_df['recency'].rank(method='first'),
                             q=5, labels=[5, 4, 3, 2, 1]).astype(int)

# Tính F_score (1-5)
# Frequency càng lớn càng tốt
rfm_df['F_score'] = pd.qcut(rfm_df['frequency'].rank(method='first'),
                             q=5, labels=[1, 2, 3, 4, 5]).astype(int)

# Tính M_score (1-5)
# Monetary càng lớn càng tốt
rfm_df['M_score'] = pd.qcut(rfm_df['monetary'].rank(method='first'),
                             q=5, labels=[1, 2, 3, 4, 5]).astype(int)
```

Ý nghĩa của Quintile:

- **Score 5:** Top 20% tốt nhất
- **Score 4:** 20-40%
- **Score 3:** 40-60%
- **Score 2:** 60-80%
- **Score 1:** Bottom 20%

Bước 2: Kết hợp thành RFM Score

```
# RFM Score = R_score × 100 + F_score × 10 + M_score
rfm_df['RFM_score'] = (rfm_df['R_score'].astype(str) +
                       rfm_df['F_score'].astype(str) +
                       rfm_df['M_score'].astype(str)).astype(int)

print(rfm_df[['customer_id', 'R_score', 'F_score', 'M_score', 'RFM_score']])
```

Ví dụ:

- **RFM_score = 555** → R=5, F=5, M=5 → **Khách hàng tốt nhất**
- **RFM_score = 111** → R=1, F=1, M=1 → **Khách hàng kém nhất**
- **RFM_score = 451** → R=4 (gần đây), F=5 (nhiều giao dịch), M=1 (giá trị thấp)

3.2. Phân nhóm khách hàng

Dựa trên RFM Score, phân khách hàng thành các nhóm:

```
def assign_segment(rfm_df):
    """Phân nhóm khách hàng dựa trên RFM Score"""
    def get_segment(row):
        r, f, m = row['R_score'], row['F_score'], row['M_score']

        if r >= 4 and f >= 4 and m >= 4:
            return 'Champions'
        elif r >= 3 and f >= 3 and m >= 3:
            return 'Loyal Customers'
        elif r >= 4 and f <= 2:
            return 'New Customers'
        elif r >= 3 and f >= 3 and m <= 2:
            return 'Potential Loyalists'
        elif r <= 2 and f >= 3:
            return 'Need Attention'
        elif r <= 2 and f <= 2 and m >= 3:
            return 'About to Sleep'
        elif r <= 2 and f <= 2 and m <= 2:
            return 'Lost'
        else:
            return 'At Risk'

    rfm_df['segment'] = rfm_df.apply(get_segment, axis=1)
    return rfm_df

rfm_df = assign_segment(rfm_df)
```

Bảng phân nhóm chi tiết:

RFM Score	Nhóm	Đặc điểm	Chiến lược Marketing
555, 554, 545, 544	Champions	R cao, F cao, M cao	VIP program, giữ chân
543, 542, 541, 535, 534, 533, 532, 531	Loyal Customers	F cao, M cao	Upsell, cross-sell
525, 524, 523, 522, 521, 515, 514, 513, 512, 511	Potential Loyalists	R cao, F trung bình	Tăng tần suất sử dụng
455, 454, 445, 444, 435, 434, 433, 432, 431	New Customers	R cao, F thấp	Onboarding, activation
355, 354, 345, 344, 335, 334, 333, 332, 331	Promising	R trung bình, F trung bình	Engagement campaigns
255, 254, 245, 244, 235, 234, 233, 232, 231	Need Attention	R thấp, F cao	Win-back campaigns

RFM Score	Nhóm	Đặc điểm	Chiến lược Marketing
155, 154, 145, 144, 135, 134, 133, 132, 131	About to Sleep	R thấp, F trung bình	Re-engagement
55, 54, 45, 44, 35, 34, 33, 32, 31	At Risk	R thấp, F thấp	Win-back hoặc chấp nhận churn
25, 24, 23, 22, 21, 15, 14, 13, 12, 11	Lost	R rất thấp, F thấp	Chấp nhận churn hoặc win-back đặc biệt

3.3. Ưu điểm

- ☑ **Dễ hiểu và giải thích:** Logic rõ ràng, dễ truyền đạt cho non-technical stakeholders
- ☑ **Không cần chuẩn hóa:** Dựa trên ranking (quintile) nên không bị ảnh hưởng bởi outliers
- ☑ **Nhanh và đơn giản:** Tính toán nhanh, không cần ML model
- ☑ **Có tiêu chuẩn ngành:** Có bảng phân nhóm chuẩn (Champions, Loyal Customers, etc.)
- ☑ **Dễ triển khai trong database:** Có thể tính bằng SQL
- ☑ **Kết quả nhất quán:** Cùng dữ liệu → Cùng kết quả (deterministic)

3.4. Nhược điểm

- ✗ **Phân nhóm cứng nhắc:** Chỉ có 5 nhóm cho mỗi chỉ số → 125 tổ hợp ($5 \times 5 \times 5$), nhưng chỉ phân thành 8-9 segment chính
- ✗ **Không tận dụng hết thông tin:** Chỉ dựa trên ranking, bỏ qua khoảng cách thực tế giữa các giá trị
- ✗ **Phụ thuộc vào phân phối dữ liệu:** Quintile phụ thuộc vào phân phối hiện tại → Khó so sánh giữa các thời kỳ
- ✗ **Không tự động tìm patterns:** Phải định nghĩa trước các segment
- ✗ **Có thể bỏ sót cluster phức tạp:** Không phát hiện được mối quan hệ phi tuyến giữa R, F, M
- ✗ **Số lượng segment cố định:** Khó điều chỉnh số lượng segment theo đặc thù business

3.5. Case ứng dụng

Case 1: E-commerce - Chiến lược Email Marketing

Tình huống: Shop online có 10,000 khách hàng, muốn tối ưu email marketing với ngân sách hạn chế.

Giải pháp:

1. Tính RFM Score cho toàn bộ khách hàng
2. Phân nhóm: Champions, Loyal Customers, At Risk, Lost
3. **Chiến lược:**
 - **Champions:** Gửi email VIP, early access sales

- **Loyal Customers:** Upsell sản phẩm cao cấp
- **At Risk:** Win-back campaign với voucher 20%
- **Lost:** Survey về lý do rời bỏ

Kết quả: Tăng tỷ lệ mở email 15%, tăng conversion rate 8%

Case 2: Banking - Xác định Hạn mức Tín dụng

Tình huống: Ngân hàng muốn tự động xác định hạn mức tín dụng cho khách hàng hiện tại.

Giải pháp:

1. Tính RFM từ lịch sử giao dịch (6 tháng)
2. Phân nhóm theo RFM Score
3. **Quy tắc hạn mức:**
 - **Champions (555-544):** Hạn mức cao, lãi suất ưu đãi
 - **Loyal Customers:** Hạn mức trung bình-cao
 - **Potential Loyalists:** Hạn mức trung bình
 - **At Risk:** Hạn mức thấp hoặc từ chối

Kết quả: Tự động hóa 60% hồ sơ, giảm thời gian xử lý từ 3 ngày → 1 ngày

4. Phương pháp 2: K-means Clustering

4.1. Tổng quan về K-means

K-means là thuật toán clustering (phân cụm) không giám sát, tự động nhóm các điểm dữ liệu tương tự nhau vào cùng một cluster.

Nguyên lý:

1. Chọn K cluster centers (tâm cụm) ban đầu
2. Gán mỗi điểm dữ liệu vào cluster gần nhất
3. Cập nhật vị trí tâm cụm (trung bình của các điểm trong cluster)
4. Lặp lại bước 2-3 cho đến khi hội tụ

Mục tiêu: Minimize tổng khoảng cách bình phương từ các điểm đến tâm cluster (Within-Cluster Sum of Squares - WCSS)

4.2. Chuẩn hóa dữ liệu RFM

Tại sao cần chuẩn hóa?

RFM có đơn vị khác nhau:

- **Recency:** Ngày (0-365)
- **Frequency:** Số lần (1-100+)
- **Monetary:** Tiền (1,000 - 100,000,000+)

Nếu không chuẩn hóa, **Monetary** sẽ chi phối kết quả clustering vì có giá trị lớn nhất.

Min-Max Scaling

```
from sklearn.preprocessing import MinMaxScaler

# Chuẩn hóa RFM về khoảng [0, 1]
scaler = MinMaxScaler()
rfm_normalized = scaler.fit_transform(rfm_df[['recency', 'frequency',
'monetary']])

rfm_scaled = pd.DataFrame(rfm_normalized,
                           columns=['recency_norm', 'frequency_norm',
'monetary_norm'],
                           index=rfm_df.index)

rfm_df = pd.concat([rfm_df, rfm_scaled], axis=1)
```

Z-score Normalization (Standardization)

```
from sklearn.preprocessing import StandardScaler

# Chuẩn hóa về mean=0, std=1
scaler_z = StandardScaler()
rfm_zscore = scaler_z.fit_transform(rfm_df[['recency', 'frequency', 'monetary']])

rfm_scaled = pd.DataFrame(rfm_zscore,
                           columns=['recency_z', 'frequency_z', 'monetary_z'],
                           index=rfm_df.index)
```

Lưu ý với Recency:

- Recency càng nhỏ càng tốt → Có thể đảo ngược trước khi scaling:

```
# Đảo ngược Recency (giá trị lớn = tốt hơn)
rfm_df['recency_inverted'] = rfm_df['recency'].max() - rfm_df['recency']
```

4.3. Xác định số cluster (K)

Có nhiều phương pháp để chọn K tối ưu:

4.3.1. Elbow Method

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Tính WCSS cho các giá trị K từ 2 đến 10
```

```
wcss = []
K_range = range(2, 11)

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(rfm_scaled)
    wcss.append(kmeans.inertia_)

# Vẽ Elbow curve
plt.figure(figsize=(10, 6))
plt.plot(K_range, wcss, 'bo-')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('WCSS (Within-Cluster Sum of Squares)')
plt.title('Elbow Method for Optimal K')
plt.grid(True)
plt.show()
```

Cách chọn: Chọn K tại "khủy tay" (elbow) của đường cong - nơi WCSS giảm chậm lại.

4.3.2. Silhouette Score

```
from sklearn.metrics import silhouette_score

silhouette_scores = []

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    cluster_labels = kmeans.fit_predict(rfm_scaled)
    silhouette_avg = silhouette_score(rfm_scaled, cluster_labels)
    silhouette_scores.append(silhouette_avg)

# Vẽ Silhouette scores
plt.figure(figsize=(10, 6))
plt.plot(K_range, silhouette_scores, 'ro-')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('Silhouette Score')
plt.title('Silhouette Score for Optimal K')
plt.grid(True)
plt.show()
```

Cách chọn: Chọn K có **Silhouette Score cao nhất** (càng gần 1 càng tốt, thường > 0.3 là chấp nhận được).

4.4. Phân nhóm bằng K-means

```
# Giả sử chọn K = 5 (dựa trên Elbow method hoặc Silhouette score)
k = 5
kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)

# Fit model
```

```
rfm_df['cluster'] = kmeans.fit_predict(rfm_scaled)

# Xem kết quả
print(rfm_df.groupby('cluster').agg({
    'recency': ['mean', 'std'],
    'frequency': ['mean', 'std'],
    'monetary': ['mean', 'std'],
    'customer_id': 'count'
}).round(2))
```

4.5. Giải thích và đặt tên cluster

Sau khi clustering, cần phân tích đặc điểm từng cluster để đặt tên phù hợp:

```
# Phân tích đặc điểm từng cluster
cluster_profile = rfm_df.groupby('cluster').agg({
    'recency': 'mean',
    'frequency': 'mean',
    'monetary': 'mean',
    'customer_id': 'count'
}).round(2)

# So sánh với trung bình tổng thể
overall_mean = rfm_df[['recency', 'frequency', 'monetary']].mean()

# Đặt tên cluster dựa trên đặc điểm
def assign_cluster_name(row):
    r, f, m = row['recency'], row['frequency'], row['monetary']

    # Champions: R thấp, F cao, M cao
    if r < overall_mean['recency'] and f > overall_mean['frequency'] and m > overall_mean['monetary']:
        return 'Champions'
    # At Risk: R cao, F thấp, M thấp
    elif r > overall_mean['recency'] and f < overall_mean['frequency'] and m < overall_mean['monetary']:
        return 'At Risk'
    # ... các điều kiện khác

    return f'Cluster_{row.name}'

# Gán tên cho cluster (cần điều chỉnh logic theo kết quả thực tế)
```

4.6. Ưu điểm

- ☑ **Tự động tìm patterns:** Không cần định nghĩa trước các segment, thuật toán tự phát hiện
- ☑ **Linh hoạt số lượng cluster:** Có thể chọn K tùy theo nhu cầu business (3-10 clusters)
- ☑ **Tận dụng toàn bộ thông tin:** Sử dụng giá trị thực tế, không chỉ ranking

- ☑ **Phát hiện cluster phức tạp:** Có thể phát hiện mối quan hệ phi tuyến giữa R, F, M
- ☑ **Tối ưu hóa khoảng cách:** Cluster được tối ưu để minimize within-cluster variance
- ☑ **Có thể mở rộng:** Dễ thêm các features khác (nếu có) vào clustering

4.7. Nhược điểm

- ✗ **Cần chuẩn hóa:** Phải chuẩn hóa dữ liệu, nhạy cảm với outliers
- ✗ **Phụ thuộc vào K:** Kết quả phụ thuộc vào số cluster chọn (K), cần test nhiều giá trị
- ✗ **Không deterministic:** Mỗi lần chạy có thể cho kết quả khác (random initialization)
- ✗ **Cần giải thích:** Phải phân tích đặc điểm cluster để đặt tên, không có bảng chuẩn
- ✗ **Tốn thời gian hơn:** Tính toán phức tạp hơn RFM Score
- ✗ **Khó triển khai trong SQL:** Cần Python/R hoặc ML platform
- ✗ **Có thể cho kết quả không có ý nghĩa business:** Cluster có thể không align với logic business

4.8. Case ứng dụng

Case 1: SaaS Platform - Phân nhóm User theo Mức độ Engagement

Tình huống: SaaS có 50,000 users, muốn phân nhóm để tối ưu onboarding và retention.

Giải pháp:

1. Tính RFM từ dữ liệu login và usage (3 tháng)
 - **R:** Ngày từ lần login cuối
 - **F:** Số lần sử dụng feature
 - **M:** Giá trị subscription (USD/month)
2. Chuẩn hóa bằng StandardScaler
3. Dùng Elbow method → Chọn K=6
4. K-means clustering

Kết quả 6 clusters:

- **Cluster 0:** Power Users (R thấp, F rất cao, M cao) → Early access features
- **Cluster 1:** At Risk (R cao, F thấp, M trung bình) → Re-engagement email
- **Cluster 2:** Champions (R thấp, F cao, M cao) → VIP support
- **Cluster 3:** Free Riders (R trung bình, F thấp, M thấp) → Upsell campaigns
- **Cluster 4:** New Users (R rất thấp, F thấp, M thấp) → Onboarding sequence
- **Cluster 5:** Dormant (R rất cao, F=0) → Win-back hoặc churn

Kết quả: Tăng retention rate 12%, giảm churn 8%

Case 2: Telecom - Phân nhóm Thuê bao để Tối ưu Gói cước

Tình huống: Nhà mạng có hàng triệu thuê bao, muốn recommend gói cước phù hợp.

Giải pháp:

- 1. Tính RFM từ dữ liệu sử dụng (1 tháng)
 - **R**: Ngày từ lần sử dụng cuối (call/data)
 - **F**: Số ngày có sử dụng trong tháng
 - **M**: Tổng chi phí trong tháng
- 2. Dùng Robust Scaling (để xử lý outliers - heavy users)
- 3. K-means với K=7 (đúng với số lượng gói cước)

Kết quả:

- Mỗi cluster tương ứng với một nhóm gói cước phù hợp
- Automated recommendation system

Kết quả: Tăng upsell rate 20%, giảm churn 15%

5. So sánh 2 Phương pháp

5.1. Bảng so sánh chi tiết

Tiêu chí	RFM Score (Quintile)	K-means Clustering
Độ phức tạp	Đơn giản	Phức tạp hơn
Tốc độ tính toán	Rất nhanh	Chậm hơn
Cần chuẩn hóa	Không	Có (bắt buộc)
Số lượng segment	Cố định (8-9 nhóm)	Linh hoạt (3-10+)
Tự động tìm patterns	Không	Có
Tận dụng thông tin	Chỉ ranking	Giá trị thực tế
Giải thích được	Dễ	Khó hơn
Có tiêu chuẩn ngành	Có (Champions, At Risk, ...)	Không
Deterministic	Có	Không (random init)
Triển khai SQL	Dễ	Khó
Xử lý outliers	Tốt (dùng ranking)	Cần Robust Scaling
Phát hiện cluster phức tạp	Không	Có
Khi nào dùng	Quick wins, reporting	Deep analysis, research

5.2. Khi nào dùng phương pháp nào?

Dùng RFM Score khi:

- ☒ Cần kết quả **nhANH** cho báo cáo định kỳ
- ☒ Team **non-technical** cần hiểu và sử dụng

- ☑ Cần **consistency** - cùng dữ liệu → cùng kết quả
- ☑ Muốn so sánh với **industry benchmarks** (Champions, Loyal Customers)
- ☑ Dữ liệu có **outliers lớn** (RFM Score ít bị ảnh hưởng)
- ☑ **Triển khai trong database** (SQL)
- ☑ Cần **quick wins** cho marketing campaigns

Dùng K-means khi:

- ☑ Cần **deep analysis** để phát hiện patterns ẩn
- ☑ Số lượng segment **linh hoạt** theo nhu cầu business
- ☑ Có **nhiều features** (không chỉ R, F, M)
- ☑ Có thời gian để **tuning và testing** (chọn K, chuẩn hóa)
- ☑ Team có **technical background** để giải thích kết quả
- ☑ Muốn **khám phá** dữ liệu, tìm insights mới
- ☑ Dữ liệu **sạch, đã xử lý outliers**

6. Chuẩn hóa Feature cho RFM

6.1. Tại sao cần chuẩn hóa?

RFM có:

- **Đơn vị khác nhau:** Ngày vs Số lần vs Tiền
- **Scale khác nhau:** Recency (0-365) vs Monetary (1,000 - 100,000,000)
- **Phân phối khác nhau:** Recency ~ uniform, Monetary ~ log-normal

Không chuẩn hóa → Monetary sẽ chi phối kết quả clustering.

6.2. Min-Max Scaling

Công thức:
$$X_{\text{normalized}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

Kết quả: Giá trị trong khoảng $[0, 1]$

Ưu điểm:

- Dễ hiểu (0-1 range)
- Giữ được mối quan hệ tương đối

Nhược điểm:

- Nhạy cảm với outliers

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
rfm_normalized = scaler.fit_transform(rfm_df[['recency', 'frequency',
'monetary']])
```

6.3. Z-score Normalization (Standardization)

Công thức:
$$X_{\text{normalized}} = \frac{X - \mu}{\sigma}$$

Kết quả: Mean = 0, Std = 1

Ưu điểm:

- Phù hợp với dữ liệu phân phối chuẩn
- Xử lý outliers tốt hơn Min-Max

Nhược điểm:

- Không có range cố định

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
rfm_zscore = scaler.fit_transform(rfm_df[['recency', 'frequency', 'monetary']])
```

6.4. Robust Scaling

Công thức:
$$X_{\text{normalized}} = \frac{X - \text{median}(X)}{\text{IQR}}$$

Kết quả: Median = 0, IQR = 1

Ưu điểm:

- **Rất ít bị ảnh hưởng bởi outliers**
- Phù hợp với dữ liệu có skew

Nhược điểm:

- Có thể cho range rộng

```
from sklearn.preprocessing import RobustScaler

scaler = RobustScaler()
rfm_robust = scaler.fit_transform(rfm_df[['recency', 'frequency', 'monetary']])
```

6.5. Khuyến nghị

- **RFM Score:** Không cần chuẩn hóa (dùng ranking)
 - **K-means với dữ liệu sạch:** Dùng **StandardScaler** (Z-score)
 - **K-means với outliers:** Dùng **RobustScaler**
-

7. Visualization

7.1. RFM Heatmap

Heatmap hiển thị phân bố khách hàng theo R và F, với màu sắc thể hiện M trung bình.

```
import seaborn as sns

# RFM Heatmap (cho RFM Score method)
heatmap_data = rfm_df.pivot_table(
    values='monetary',
    index='R_score',
    columns='F_score',
    aggfunc='mean'
)

plt.figure(figsize=(12, 8))
sns.heatmap(heatmap_data, annot=True, fmt='.0f', cmap='YlOrRd')
plt.title('RFM Heatmap: Average Monetary Value')
plt.xlabel('Frequency Score')
plt.ylabel('Recency Score')
plt.show()
```

7.2. Segment Profile

So sánh R, F, M trung bình giữa các segment.

```
# Bar chart
segment_avg = rfm_df.groupby('segment').agg({
    'recency': 'mean',
    'frequency': 'mean',
    'monetary': 'mean'
}).sort_values('monetary', ascending=False)

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

axes[0].barh(segment_avg.index, segment_avg['recency'])
axes[0].set_xlabel('Average Recency (days)')
axes[0].set_title('Average Recency by Segment')

axes[1].barh(segment_avg.index, segment_avg['frequency'])
axes[1].set_xlabel('Average Frequency')
axes[1].set_title('Average Frequency by Segment')

axes[2].barh(segment_avg.index, segment_avg['monetary'])
```



```
axes[2].set_xlabel('Average Monetary')
axes[2].set_title('Average Monetary by Segment')

plt.tight_layout()
plt.show()
```

7.3. Cluster Visualization (K-means)

```
# 3D Scatter plot cho K-means clusters
from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure(figsize=(12, 10))
ax = fig.add_subplot(111, projection='3d')

clusters = rfm_df['cluster'].unique()
colors = plt.cm.tab10(range(len(clusters)))

for i, cluster in enumerate(clusters):
    cluster_data = rfm_df[rfm_df['cluster'] == cluster]
    ax.scatter(cluster_data['recency'],
               cluster_data['frequency'],
               cluster_data['monetary'],
               label=f'Cluster {cluster}', alpha=0.6, c=[colors[i]])

ax.set_xlabel('Recency')
ax.set_ylabel('Frequency')
ax.set_zlabel('Monetary')
ax.set_title('RFM 3D Visualization by K-means Cluster')
ax.legend()
plt.show()
```

Tổng kết

Sau buổi học, bạn đã nắm được:

1. ☒ **RFM Analysis** là gì và tại sao quan trọng
2. ☒ Cách tính **R, F, M** từ dữ liệu giao dịch
3. ☒ **Phương pháp 1: RFM Score (Quintile)** - Ưu nhược điểm và case ứng dụng
4. ☒ **Phương pháp 2: K-means Clustering** - Ưu nhược điểm và case ứng dụng
5. ☒ **So sánh 2 phương pháp** và khi nào dùng phương pháp nào
6. ☒ Cách **chuẩn hóa** dữ liệu RFM
7. ☒ **Visualization** RFM và clusters

Lưu ý:

- Không có phương pháp nào "tốt nhất"
- Chọn phương pháp phù hợp với context và mục tiêu
- Có thể kết hợp cả 2 phương pháp: RFM Score cho reporting, K-means cho deep analysis

